

V93K测试机基础知识

学习目标

- 了解 V93000 测试机台的产生背景与市场地位
- 掌握 V93000 测试机的系统组成与关键硬件资源
- 理解测试向量、时序与信号周期等 ATE 基本概念
- 熟悉 STIL 测试程序与 test method 编程模型
- 掌握测试程序的调试、执行与并行测试优化

1 V93000 测试机背景

V93000 是爱德万测试（Advantest）推出的旗舰级 SoC 测试系统，业内常简称为 V93K。它是一台大规模集成电路自动测试设备（ATE，Automatic Test Equipment），用于在芯片制造完成后对其电气特性、功能与可靠性进行自动化验证。V93000 凭借每引脚独立（Per-Pin）架构与灵活的板卡组合，覆盖数字、模拟、混合信号与射频等多种测试需求，广泛应用于手机处理器（AP）、基带芯片、电源管理芯片（PMIC）、图像传感器（CIS）以及微控制器（MCU）等产品的量产测试。

1.1 ATE 的基本概念

自动测试设备（ATE）是芯片测试环节的核心装备。一颗芯片从设计到量产通常经历三个阶段：

- 晶圆测试（CP，Chip Probing）：在晶圆切割前，用探针卡（Probe Card）接触晶圆上的裸片（Die），筛选出功能正常的管芯，坏片通常打墨点标记。
- 成品测试（FT，Final Test）：晶圆切割、封装完成后，对封装芯片进行最终测试，保证出货良率。
- 系统级测试（SLT，System Level Test）：模拟真实应用场景（如手机整机环境）对芯片进行功能验证，弥补 ATE 难以覆盖的场景。

ATE 的基本工作方式是：测试机通过测试程序（Test Program）生成激励信号（测试向量），加载到被测器件（DUT，Device Under Test）的引脚上，同时采集器件的响应信号，与期望值进行比较，从而判断器件好坏（Pass/Fail）。

1.2 V93000 的产品定位

V93000 采用模块化的板卡架构，同一台测试头（Test Head）上可以混插不同功能的板卡，从而组合出面向不同产品的测试解决方案。其主要竞争对手是泰瑞达（Teradyne）的 UltraFLEX 系列，二者共同占据全球 SoC 测试机市场的大部分份额。V93000 的典型应用领域如下表所示：

应用领域	典型被测芯片	常用板卡资源
手机应用处理器	AP、SoC	PS1600 数字板卡
通信芯片	基带、射频收发机	PS1600 + AVI64（RF）
电源管理	PMIC	直流电源板卡（DPS、SMU）

应用领域	典型被测芯片	常用板卡资源
传感器与图像	MEMS、CIS	低速数字 + 模拟板卡

2 V93000 系统组成

V93000 测试系统主要包括测试头（Test Head）、主机工作站（Workstation）、冷却单元（Cooling Unit）与操纵台（Manipulator）等部分。其中测试头是系统的核心，内部插满各种测试板卡（Board）。

2.1 测试头与板卡

测试头内部的关键板卡包括：

- 数字通道板卡（如 Pin Scale 1600，简称 PS1600）：提供高速数字测试通道，是测试机的核心资源。每个通道独立产生激励并采样响应，通道之间互不干扰。
- 模拟板卡（如 AVI64）：提供任意波形发生器（AWG）与数字采样器（Digitizer），用于模拟信号与混合信号测试。
- 电源板卡（DPS, Device Power Supply）：为被测器件提供可编程的供电电源，并具备电流钳制保护能力。
- 控制板卡（Master Controller）：运行测试程序，协调所有板卡的工作。

Per-Pin 架构是 V93000 最核心的设计思想：每一个测试通道都拥有独立的驱动、比较、时序与波形资源。这意味着不同通道可以同时运行不同频率、不同格式的测试向量，测试资源分配非常灵活，也便于通过增加板卡来扩展通道数量。

2.2 测试程序的结构

一套完整的 V93000 测试程序由两类文件组成：

1. 器件定义文件（Device Definition）：使用 STIL（Standard Test Interface Language）语言描述被测器件的信息，包括引脚定义、信号电平、时序、测试向量（Pattern）以及测试条件等。
2. 测试方法文件（Test Method）：使用 C++ 编写，通常为一个 TestClass。测试方法通过调用测试 API（如 Functional Test API、DC Test API 等）来执行具体的测试项，并判定测试结果。

简单理解：STIL 文件回答“测什么、用什么电平与时序去测”，Test Method 回答“如何调用资源执行测试并给出结论”。

3 测试向量与基本概念

3.1 测试向量

测试向量（Test Vector 或 Pattern）是加载到器件引脚上的一组激励信号序列。一个测试向量描述了某一个时刻（周期）内所有相关引脚的状态。例如，对一个带时钟输入 clk、复位输入 rst_n、数据输入 din 和输出 dout 的简单数字器件，测试向量可以表示为：

周期	clk	rst_n	din	dout(期望)
1	0	0	0	X
2	1	0	1	X

周期	clk	rst_n	din	dout(期望)
3	0	0	0	1
4	0	0	0	0

其中 0/1 表示高低电平，X 表示该周期不检查（Don't Care）。测试机在每个周期按照向量给出的值驱动输入引脚，并将输出引脚采样到的实际值与向量中的期望值比较：一致则 Pass，不一致则 Fail。

3.2 信号周期与边沿

测试机的数字通道按信号周期（Period）循环工作。一个周期内可以定义多个边沿事件，每个边沿事件指定了动作发生的时间点。关键的时间点包括：

- 驱动边沿（Drive Edge）：测试机在此时刻将激励信号驱动到引脚上。
- 比较边沿（Compare Edge）：测试机在此时刻对引脚信号进行采样并与期望值比较。

边沿时间通常用纳秒（ns）表示。例如周期 10ns、驱动边沿 0ns、比较边沿 7ns，表示每 10ns 循环一次，周期开始即驱动，第 7ns 时采样比较。边沿定位精度（Edge Placement Accuracy）是衡量测试机数字性能的重要指标之一。

3.3 波形的数据格式

测试机用波形格式（Data Format）描述一个周期内信号的形态。常见的格式包括：

- NRZ（Non-Return to Zero）：信号在周期内保持电平不变，是最常用的格式。
 - RZ（Return to Zero）：信号在周期内先驱动到指定电平，再回到 0 电平。
 - R0（Return to One）：与 RZ 类似，但回到 1 电平。
 - SBC（Surround By Complement）：信号先驱动到补码电平，再驱动到指定电平，可以增强信号边沿能量。
- 格式的选择会影响信号质量与测试稳定性，一般数字测试默认使用 NRZ 格式。

3.4 时序文件示例

V93000 的 STIL 器件定义文件中，时序通过 WFC（Waveform Character）与 Timing 定义来描述。下面是一个简化的示例片段：

```
STIL 1.0;
Signals {
    clk In;
    din In;
    dout Out;
}
SignalGroups {
    _pi = 'clk + din';
    _po = 'dout';
}
Timing {
    WaveformTable main_wft {
        Period '10ns';
        Waveforms {
            _pi { 01 { '0ns' D; } 10 { '0ns' D; } }
            _po { H { '7ns' C; } L { '7ns' C; } }
        }
    }
}
```

```
}  
}
```

上述定义表示：输入引脚组 `_pi` 的周期为 10ns，在第 0ns 驱动；输出引脚组 `_po` 在第 7ns 采样比较。

4 SmarTest 测试程序编程

4.1 开发环境

V93000 的软件环境称为 SmarTest，运行在主机工作站上。SmarTest 提供图形化的编程与调试界面，支持器件定义文件的编辑、测试方法的编译、测试数据的采集与显示等。测试工程师在 SmarTest 中完成测试程序的开发与调试后，将程序部署到产线测试机台上执行量产测试。

4.2 测试方法 (Test Method)

测试方法是用 C++ 编写的测试类，核心是若干成员函数。每一个测试项对应一个独立的 test method。一个最简单的测试方法框架如下：

```
#include "V93kTestAPI.h"  
  
class BasicTest : public TestClass {  
public:  
    // 测试流程设置，只在程序加载时执行一次  
    void setup() {  
        // 在这里设置测试参数  
    }  
  
    // 真正的测试执行体，每测试一颗芯片都会执行  
    void execute() {  
        // 调用功能测试 API，执行测试向量  
        FunctionalTest functest;  
        functest.pinList("@");  
        functest.execute();  
  
        // 判断结果  
        if (functest.hasPassed()) {  
            cout << "Test PASS" << endl;  
        } else {  
            cout << "Test FAIL" << endl;  
        }  
    }  
};  
  
// 注册测试方法，使 SmarTest 能够识别  
REGISTER_TESTCLASS("basic_test", BasicTest);
```

4.3 常用测试 API

测试方法中通过各类测试 API 完成具体测试：

- **FunctionalTest**：功能测试，执行 STIL 中定义的测试向量。
- **DC_Test**：直流参数测试，如开路/短路测试 (Continuity)、漏电流测试等。
- **Analog_Test**：模拟测试，调用 AWG 产生波形、Digitizer 采样分析。

- T_Test: 时间测量, 如频率测量、传播延时测量。

测试方法中还可以通过 pinList 指定参与测试的引脚, 例如 pinList("@") 表示使用器件定义文件中的全部引脚。

5 测试程序的调试与执行

5.1 测试流程

测试程序通常按照测试流程 (Test Flow) 组织, 流程中的节点包括:

- 环境初始化: 加载测试程序, 校准板卡, 设置测试条件。
- 按顺序执行测试项: 先做开路/短路测试 (Contact Test), 再做功能测试、直流测试等。
- 结果记录: 每个测试项的结果写入数据日志 (Datalog), 用于后续良率分析。

5.2 常见调试手段

调试测试程序时常用以下手段:

- 示波器 (Scope) 工具: 观察引脚上的实际波形, 检查信号质量。
- Shmoo 测试: 对电压、频率等参数进行二维扫描, 观察器件在参数变化下的表现, 寻找器件工作边界。
- Datalog 分析: 查看失败测试项的失败周期与失败引脚, 定位是器件问题还是测试程序问题。
- 波形显示: 将测试向量以波形图形式展开, 直观检查向量与时序定义。

5.3 常见问题排查

- 量产良率突然下降: 优先检查探针卡/插座接触、测试程序版本、硬件板卡状态是否发生变化。
- 某引脚持续失败: 检查该引脚在器件定义文件中的电平、时序定义, 以及测试头上的信号链路。
- 测试结果不稳定: 检查信号边沿时间设置是否过于临界, 供电是否稳定, 必要时放宽时序或增加采样次数。

6 并行测试与产能优化

6.1 多工位并行测试

多工位 (Multi-Site) 测试是提高测试产能 (Throughput) 的主要手段。同一台测试机上同时测试多颗芯片, 例如 x2、x4、x8 工位。V93000 的 Per-Pin 架构天然支持多工位并行: 每颗芯片占用独立的通道资源, 各工位之间互不干扰。

6.2 产能计算

测试产能通常用 UPH (Units Per Hour, 每小时测试芯片数) 衡量。计算公式为:

$$UPH = 3600 / T_{avg}$$

其中 T_{avg} 为单颗芯片的平均测试时间 (单位: 秒)。例如单颗芯片测试时间为 3.6 秒, 则:

$$UPH = 3600 / 3.6 = 1000$$

即每小时可测试 1000 颗芯片。若采用 x4 工位并行测试, 且测试时间不变, 则等效产能为 4000 颗/小时。

6.3 产能优化方法

- 减少测试项：通过良率数据分析去掉对最终质量贡献小的测试项，缩短测试时间。
- 测试项并发执行：利用 Per-Pin 架构的独立性，让相互独立的测试项并行运行。
- 优化测试向量：压缩向量长度，减少冗余周期。
- 优化 Index 时间：缩短上下料、分选（Handler）与测试之间的衔接时间。

7 小结总结

本讲义主要介绍 V93000 测试机的背景知识与基本概念：

- 了解 ATE 在芯片测试流程（CP/FT/SLT）中的作用，以及 V93000 的 Per-Pin 架构与模块化板卡设计。
- 掌握测试程序的两大组成部分：STIL 器件定义文件与 C++ 测试方法（Test Method）。
- 理解测试向量、信号周期、驱动边沿、比较边沿、波形数据格式等基本概念。
- 熟悉 FunctionalTest、DC_Test 等常用测试 API 的使用方式。
- 掌握调试手段（Shmoo、Datalog、示波器）与并行测试产能优化方法。